

Final Capstone Report

Retail Data Platform — AWS Lakehouse + Databricks Parallel Consumption

1. Architecture Diagram

The diagram below traces the full pipeline: synthetic data generation, ingestion into an S3 medallion lakehouse, cataloging and governed access, and two parallel downstream consumption paths — Amazon Redshift Serverless for structured order/customer/product analytics, and Databricks for clickstream ingestion and dashboarding.

End-to-End Capstone Architecture

Synthetic Data → Lakehouse (S3) → Governed Catalog → Parallel Analytical Consumption

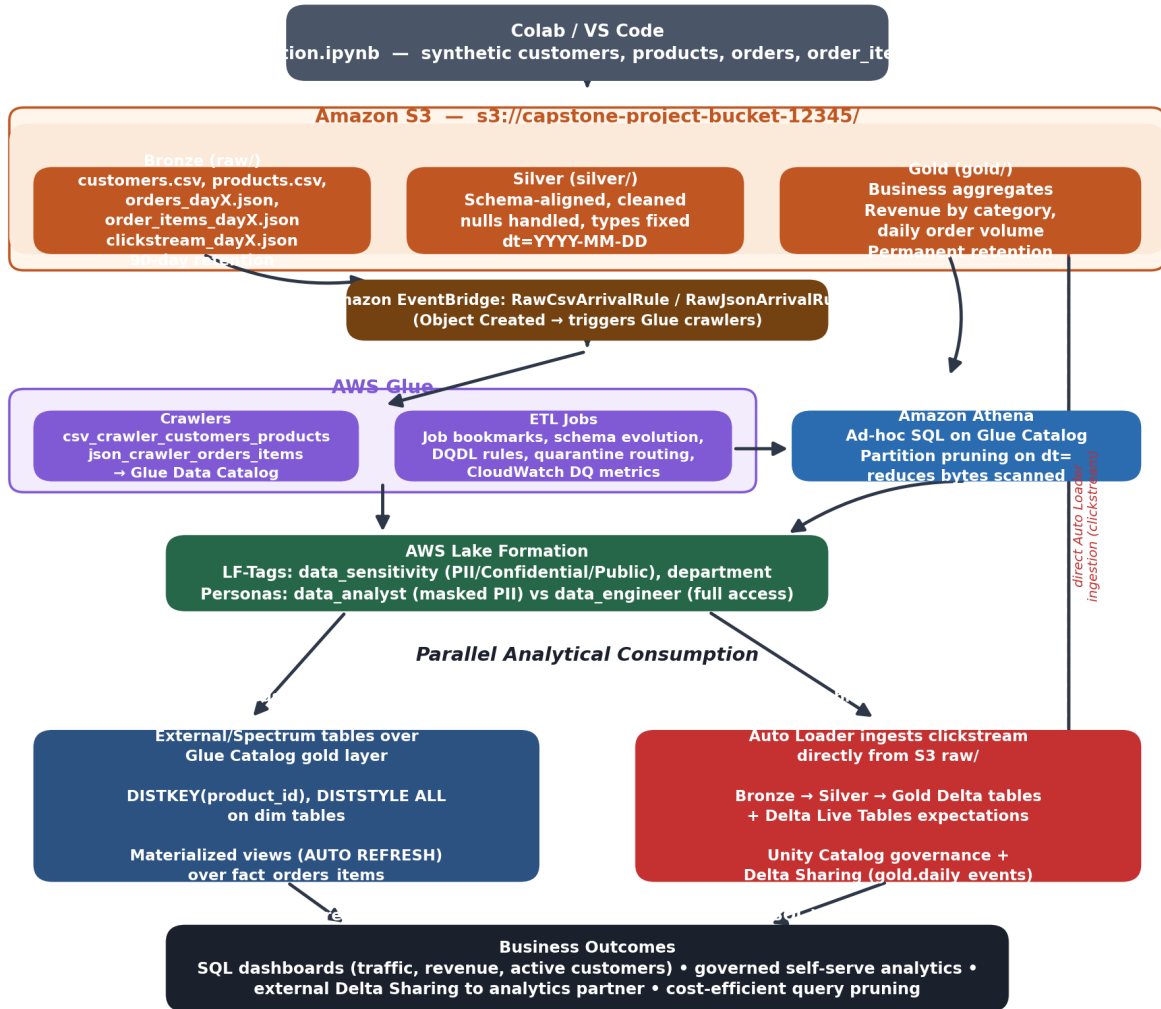


Figure 1. End-to-end capstone architecture: Colab/VS Code → S3 (Bronze/Silver/Gold) → Glue Catalog/Athena → Lake Formation → Redshift Serverless and Databricks in parallel.

2. Cost Review

2.1 What was provisioned

- Amazon S3 bucket (**capstone-project-bucket-12345**) with raw/, silver/, gold/ prefixes, Hive-style **dt=YYYY-MM-DD** partitioning, and a 90-day lifecycle rule on the raw zone only.
- Two AWS Glue crawlers (**csv_crawler_customers_products**, **json_crawler_orders_items**) and Glue ETL jobs with job bookmarks, schema-evolution handling, and a DQDL data-quality ruleset with quarantine routing.
- AWS Glue Data Catalog holding a handful of tables (customers, products, orders, order_items) — well within the always-free 1M-object allowance.
- Two Amazon EventBridge rules (RawCsvArrivalRule, RawJsonArrivalRule) watching S3 object-creation events.
- AWS Lake Formation governance: bucket registration, two LF-Tag keys (data_sensitivity, department), and two IAM personas (data_analyst, data_engineer) with differentiated column-level grants.
- Amazon Athena, queried ad hoc against the Glue Catalog (no persistent compute, pay-per-query).
- Amazon Redshift Serverless with external/Spectrum tables over the gold layer, a DISTKEY(product_id) fact table, DISTSTYLE ALL dimension table, and an auto-refreshing materialized view.
- A Databricks workspace: a small job/all-purpose cluster (single node, 1 × 32 GB gp3 EBS volume) running Auto Loader, the Bronze/Silver/Gold Delta pipeline, a Delta Live Tables pipeline, Unity Catalog, Delta Sharing, and a SQL Warehouse dashboard.
- Custom Amazon CloudWatch metrics (OrdersValidCount, OrdersQuarantineCount) in a CapstoneDQ namespace.

2.2 What was torn down / never left running

- Databricks clusters used auto-termination on inactivity, so no idle All-Purpose compute (the most expensive DBU tier) accrued charges between sessions.
- Redshift Serverless has no fixed cluster to shut down — RPU-hours are only billed while a query is actively executing, so cost fell to zero between test runs.
- Glue crawlers and ETL jobs are on-demand by design; no development endpoints or standing Spark clusters were left provisioned.
- No EC2 instances, NAT gateways, or Redshift Provisioned clusters were created outside of what Databricks and Redshift Serverless manage internally.
- S3 raw data is time-boxed to 90 days via lifecycle policy; only the small gold aggregates are retained permanently.

2.3 Actual cost profile at capstone scale

Data volumes in this project were intentionally small (~5,000 customers, ~800 products, ~24,000 orders, ~66,000 order items, ~30,000 clickstream events across two days — see [data_profile_report.md](#)). Athena scans measured in the KB–MB range per query ([athena_cost_comparison.md](#)), Glue jobs run in seconds to low minutes, and the Databricks cluster ran only during active development. At this scale the entire stack sits inside AWS/Databricks free-tier and minimum-billing thresholds — realistically a few dollars per month, driven mostly by the Redshift Serverless 4-RPU minimum activation charge (~\$1.50 per active hour) and the Databricks EBS/EC2 minutes actually used, rather than by data volume.

2.4 Projected cost at 10× data volume (~300K clickstream events, ~660K order items, proportional growth elsewhere)

Component	Driver	Approx. rate	Est. monthly at 10×
S3 storage (raw+silver+gold)	GB stored, Standard tier	~\$0.023/GB-mo	< \$1
Glue crawlers + ETL jobs	DPU-hours per run	\$0.44/DPU-hr	\$3 – \$8

Glue Data Catalog	Objects stored	free < 1M objects	\$0 (still free tier)
Amazon Athena	TB scanned per query	\$5.00/TB	< \$1 (partition pruning keeps scans small)
AWS Lake Formation	No direct usage fee	\$0	\$0
Amazon EventBridge	Events published	free < 1M events/mo	\$0
Redshift Serverless	RPU-hours while active (4-RPU min.)	~\$1.50/hr min.	\$45 – \$90 (assumes 1–2 active hrs/day)
Databricks DBU (Jobs Compute)	DBUs per job run, Premium tier	~\$0.15/DBU	\$10 – \$20
Databricks underlying EC2 + EBS	Instance-hours + GB-month gp3	instance + ~\$0.08/GB-mo	\$25 – \$45
CloudWatch custom metrics	Metric count	~\$0.30/metric-mo	< \$1

Estimated total at 10× volume: roughly \$85 – \$165/month. These are list-price, illustrative estimates (US region, on-demand pricing as referenced from current AWS and Databricks pricing pages) based on the provisioning documented above — not figures pulled from an actual billing export. The dominant cost driver is **compute activation time** (Redshift Serverless RPU-hours and the Databricks cluster's EC2/DBU minutes), not raw data volume, because every component here is serverless or on-demand and partition pruning keeps bytes-scanned low even as row counts grow. The clearest lever to control cost at higher scale is minimizing how long the Redshift Serverless warehouse and Databricks clusters stay active per day, rather than optimizing storage.

3. Reflection

If this were a real production system, the design decision I would change is running **two separate governance systems** for what is ultimately one logical dataset: AWS Lake Formation (LF-Tags, data_analyst / data_engineer personas) governing the Glue Catalog / Athena / Redshift Spectrum path, and Databricks Unity Catalog governing the clickstream Delta tables ingested directly from S3 via Auto Loader.

In this capstone the split was harmless because the two paths cover different source tables (orders/customers/products vs. clickstream) and were built independently to demonstrate each tool. In production, though, dual governance layers create a real risk: PII or sensitivity classifications get defined twice, in two different tagging systems, by two different teams. LF-Tags marking a column as PII in Lake Formation give no guarantee that the equivalent Unity Catalog table (or a future table that joins clickstream data with customer identity) carries the same classification. Access reviews, audits, and incident response all have to check two systems instead of one, and it is easy for a policy change in one catalog to silently drift out of sync with the other.

The change I'd make: pick a single source of truth for data classification and access policy up front. Concretely, either (a) register both the AWS-native tables and the Databricks external Delta tables under one Unity Catalog metastore with Lakehouse Federation, so PII tags, masking, and row/column-level rules are defined once and enforced everywhere, or (b) if Lake Formation must remain the system of record for the AWS-native path, mirror only the classification tags (not the enforcement logic) into Unity Catalog via an automated sync job rather than re-declaring them by hand. Either approach removes the class of bug where "the data is protected in one place but not the other" — which is exactly the kind of gap that shows up in a real audit, not in a capstone demo.